

(Applications Development and Emerging Technologies)

TECHNICAL-SUMMATIVE ASSESSMENT

3

USER INTERFACES, SESSION AND COOKIES, MYSQL DATABASE AND RESET PASSWORD FEATURE

Student Name / Group Name:	Francine Agalabia	
Members (if Group):	Name	Role
Section:	TC23	
Professor:	Mr. Calleja	

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements.

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Understand and apply best practices and standards in the development of website.

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- To apply the available super global variables for form processing and validation.
- To differentiate the use of \$_GET, \$_POST, and \$_REQUEST super global variable in form processing and know when to use it.
- To differentiate the use of Session and Cookies for form security of a Web Site.
- To know the proper syntax for validating user inputs using Regular Expression.
- To provide a good background of Relational Database using MySQL.
- To know the importance of Database in Web Application using MySQL.
- To Identify the importance of Database Structure in constructing tables.
- To be familiar with the syntax in managing users and database.
- To define a good structure of tables in a given database for data storage.
- To be familiar in the common syntax of creating database and tables and the correct data type to be used for each field.

IV. BACKGROUND INFORMATION

Superglobal variables

Example: Using GETs method

```
<form action="<?php echo $_SERVER['PHP_SELF'] ?>" method="get">
    string: <input type="text" name="str" value="get value">
    <input type="submit" name="submit" value="submit get">
</form>
Using POST method
<form action="<?php echo $_SERVER['PHP_SELF'] ?>" method="post">
    string: <input type="text" name="str" value="post value">
    <input type="submit" name="submit" value="submit post">
</form>
<?php
echo "<br/>GET value: ";
if(isset($_GET['submit'])){
    echo $_GET['str'];
}
echo "<br/>POST value: ";
if(isset($_POST['submit'])){
    echo $_POST['str'];
}
echo "<br/>REQUEST value: ";
if(isset($_REQUEST['submit'])){
    echo $_REQUEST['str'];
}
?>
```

Cookies

Example: PHPSetCookies.php

```
SET COOKIES
<?php
setcookie("xcookie", "value of x");
setcookie("ycookie", "value of y", time()+10);
setcookie("zcookie", "value of z", time()+3600);
?>
```

Output 1: cookies were set

```
DISPLAY COOKIES
Array
(
    [xcookie] => value of x
    [ycookie] => value of y
    [zcookie] => value of z
)
```

Example: PHPDisplayCookies.php

```
DISPLAY COOKIES
<?php
echo "<pre>"; print_r($_COOKIE); echo "</pre>";
?>
```

Output 2: after 10 secs

```
DISPLAY COOKIES
Array
(
    [xcookie] => value of x
    [zcookie] => value of z
)
```

Example: PHPDeleteCookies.php

```
DELETE COOKIES
<?php
setcookie("zcookie", "value of z", time()-3600);
?>
```

Output 3: delete cookies

```
DISPLAY COOKIES
Array
(
    [xcookie] => value of x
)
```

Session

Example: PHPDisplaySession.php

```
<?php
session_start();
if($_SESSION['logged']){
    echo "Welcome " . $_SESSION['name'];
    echo ' <br/> <a href="PHPUnsetSession.php">' . $_SESSION['email'] . '</a>';
    echo ' <br/> <a href="PHPDeleteSession.php">Logout</a>';
} else{
    echo ' <a href="PHPSetSession.php">Login</a>';
}
?>
```

Example: PHPDeleteSession.php

```
<?php
session_start();
session_destroy();
header('location: PHPDisplaySession.php');
```

Example: Employee Directory DB

last_name	first_name	address	city	state	zip
Blair	Dennis	204 Spruce Lane	Brookfield	MA	01506
Hernandez	Louis	68 Boston Post Road	Spencer	MA	01562
Miller	Erica	271 Baker Hill Road	Brookfield	MA	01515
Morinaga	Scott	17 Ashley Road	Brookfield	MA	01515
Picard	Raymond	1113 Oakham Road	Barre	MA	01531

Also called as **flat-file database** that stores information in a single table.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

Activity A

1. Create a registration module just like what is shown below. Using \$_GET or \$_POST get the data from the forms and display below. Integrate HTML and CSS.

Registration Form

Registration Form

Registration Successful
First Name: Francine
Last Name: Agalabia
Email: feagalabia@fit.edu.ph
Username: pransineeeee

Registration Form

Password and Confirm Password are not the same.

registration.php

```
<?php
$message = "";
if($_SERVER["REQUEST_METHOD"] == "POST")
{
    $fname = $_POST['fname'];
    $lname = $_POST['lname'];
    $email = $_POST['email'];
    $username = $_POST['username'];
    $password = $_POST['password'];
    $confirm = $_POST['confirm'];

    if($password == $confirm)
    {
        $message = "
        <h3>Registration Successful</h3>

        First Name: $fname <br>
        Last Name: $lname <br>
        Email: $email <br>
        Username: $username
        ";
    }
    else
    {
        $message = "Password and Confirm Password are not the same.";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Activity A1</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>Registration Form</h2>
<form method="POST">
    <input type="text" name="fname" placeholder="First Name" required>
    <input type="text" name="lname" placeholder="Last Name" required>
    <input type="email" name="email" placeholder="Email" required>
    <input type="text" name="username" placeholder="Username" required>
```

```

<input type="password" name="password" placeholder="Password" required>
<input type="password" name="confirm" placeholder="Confirm Password" required>
<button type="submit">
    Register
</button>
</form>
<div class="result">
    <?php echo $message; ?>
</div>
</div>
</body>
</html>

```

style.css

```

body{
    font-family: Arial, sans-serif;
    background:#f4f4f4;
}
.container{
    width:400px;
    margin:50px auto;
    background:white;
    padding:20px;
    border-radius:10px;
}
h2{
    text-align:center;
}
input{
    width:100%;
    padding:10px;
    margin-top:10px;
}
button{
    width:100%;
    padding:10px;
    margin-top:15px;
    background:#007bff;
    color:white;
    border:none;
}
.result{
    margin-top:20px;
}

```

2. Create a login module just like what is shown below. Using `setcookie()` function, create cookies for the username and password. It must be saved in your web form if the remember me checkbox is checked only.

Login Form

Username

Password

☐ Remember Me

Login

Login Form

pransineeeee

.....

☒ Remember Me

Login

Cookies Saved!

Login Form

pransineeeee

.....

☐ Remember Me

Login

Login Form

pransineeeee

.....

☐ Remember Me

Login

login.php

```
<?php
$username = "";
$password = "";
if(isset($_COOKIE['username']))
{
    $username = $_COOKIE['username'];
}
if(isset($_COOKIE['password']))
{
    $password = $_COOKIE['password'];
}
if(isset($_POST['login']))
{
    $username = $_POST['username'];
    $password = $_POST['password'];
    if(isset($_POST['remember']))
    {
        setcookie(
            "username",
            $username,
            time() + 86400
        );
        setcookie(
            "password",
            $password,
            time() + 86400
        );
        echo "<p style='color:green;'>Cookies Saved!</p>";
    }
    else
    {
        echo "<p style='color:red;'>Remember Me not checked.</p>";
    }
}
?>

<!DOCTYPE html>
<html>

<head>
    <title>Activity A2 - Cookies</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```

<body>

<div class="container">

  <h2>Login Form</h2>

  <form method="POST">

    <input
      type="text"
      name="username"
      placeholder="Username"
      value="<?php echo $username; ?>"
      required>

    <input
      type="password"
      name="password"
      placeholder="Password"
      value="<?php echo $password; ?>"
      required>

    <label>
      <input type="checkbox" name="remember">
      Remember Me
    </label>

    <button type="submit" name="login">
      Login
    </button>

  </form>

</div>

</body>
</html>

```

style.css

```

body{
  font-family: Arial, sans-serif;
  background:#f4f4f4;
}

```

```
.container{
  width:400px;
  margin:50px auto;
  background:white;
  padding:20px;
  border-radius:10px;
}

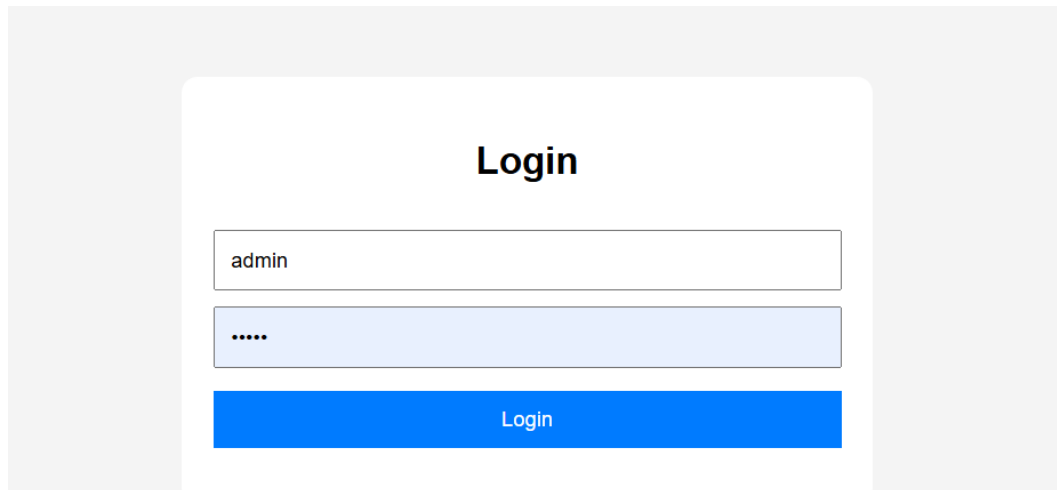
h2{
  text-align:center;
}

input[type="text"],
input[type="password"]{
  width:100%;
  padding:10px;
  margin-top:10px;
  box-sizing:border-box;
}

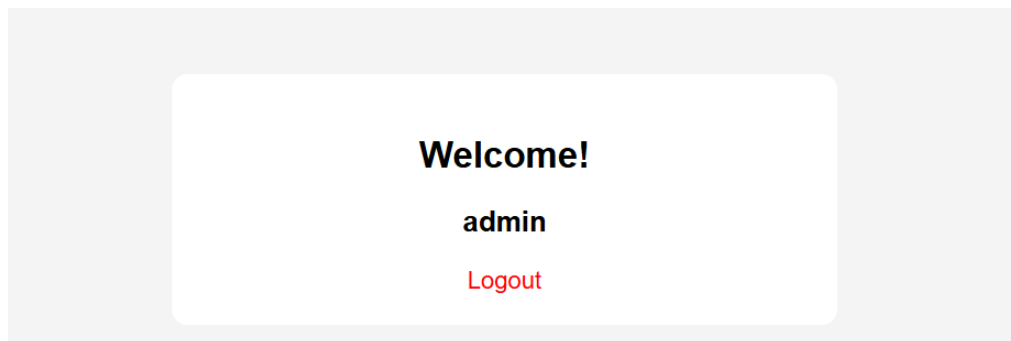
button{
  width:100%;
  padding:10px;
  margin-top:15px;
  background:#007bff;
  color:white;
  border:none;
  cursor:pointer;
}

button:hover{
  background:#0056b3;
}
```

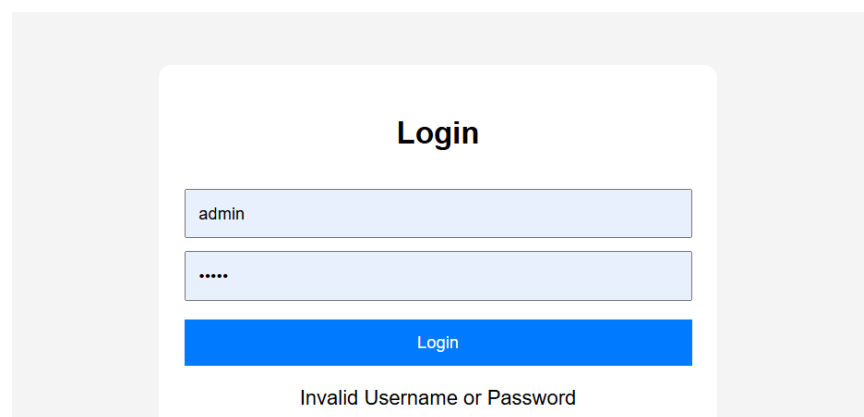
- Using the same login module, create a static variable that will hold your username and password. Get the value from the form and compare it. If the username and password matches, using `$_SESSION`, redirect the user to a homepage that will display your username stored in the session and a logout link that will redirect you to a `logout.php` containing `session_destroy()` function. If you have an active session, the user by changing the url above cannot access other pages such as the index page, login and registration pages. If there are no active sessions, users cannot open the home.php (or pages that has logout link)



A screenshot of a login form. The form is centered on a light gray background. It has a title "Login" in bold black text. Below the title are two input fields: the first contains the text "admin", and the second contains five dots. Below these fields is a blue button with the text "Login" in white.



A screenshot of a welcome page. The page is centered on a light gray background. It has a title "Welcome!" in bold black text. Below the title, the text "admin" is displayed in bold black. Below that, the text "Logout" is displayed in red.



A screenshot of a login form with an error message. The form is centered on a light gray background. It has a title "Login" in bold black text. Below the title are two input fields: the first contains the text "admin", and the second contains five dots. Below these fields is a blue button with the text "Login" in white. Below the button, the text "Invalid Username or Password" is displayed in black.

Login.php

```
<?php
session_start();
$correctUsername = "admin";
$correctPassword = "12345";

$message = "";

if(isset($_POST['login']))
{
    $username = $_POST['username'];
    $password = $_POST['password'];

    if(
        $username == $correctUsername &&
        $password == $correctPassword
    )
    {
        $_SESSION['username'] = $username;

        header("Location: home.php");
        exit();
    }
    else
    {
        $message = "Invalid Username or Password";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Activity A3 Login</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

    <h2>Login</h2>

    <form method="POST">
```

```

<input
  type="text"
  name="username"
  placeholder="Username"
  required>

<input
  type="password"
  name="password"
  placeholder="Password"
  required>

<button type="submit" name="login">
  Login
</button>

</form>

<p>
  <?php echo $message; ?>
</p>

</div>

</body>
</html>

```

```

Home.php
<?php
session_start();
if(!isset($_SESSION['username']))
{
  header("Location: login.php");
  exit();
}
?>

<!DOCTYPE html>
<html>
<head>
  <title>Home Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

```

```
<div class="container">

  <h2>Welcome!</h2>

  <h3>
    <?php echo $_SESSION['username']; ?>
  </h3>

  <a href="logout.php">
    Logout
  </a>

</div>

</body>
</html>
```

Logout.php

```
<?php

session_start();

session_destroy();

header("Location: login.php");
exit();

?>
```

Style.css

```
body{
  font-family: Arial, sans-serif;
  background:#f4f4f4;
}

.container{
  width:400px;
  margin:50px auto;
  background:white;
  padding:20px;
  border-radius:10px;
  text-align:center;
}
```

```

input{
  width:100%;
  padding:10px;
  margin-top:10px;
  box-sizing:border-box;
}

button{
  width:100%;
  padding:10px;
  margin-top:15px;
  background:#007bff;
  color:white;
  border:none;
  cursor:pointer;
}

a{
  text-decoration:none;
  color:red;
}

```

Activity B

1. Create a registration module just like what is shown below. Integrate HTML, CSS and PHP with MySQL.

Registration

Registration

Server: 127.0.0.1 » Database: sa3_db » Table: users

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)

✓ Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

`SELECT * FROM `users``

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all | Number of rows: 25 ▼ Filter rows:

Extra options

	id	firstname	lastname	email	username	password
☐ Edit ☐ Copy ☐ Delete	1	Francine	Agalabia	feagalabia@fit.edu.ph	pransineeeee	12345

```

Db.php
<?php
$conn = mysqli_connect(
    "localhost",
    "root",
    "",
    "sa3_db"
);
  
```

```
if(!$conn)
{
    die("Database Connection Failed");
}
?>
```

Registration.php

```
<?php
include("db.php");

$message = "";

if(isset($_POST['register']))
{
    $fname = $_POST['fname'];
    $lname = $_POST['lname'];
    $email = $_POST['email'];
    $username = $_POST['username'];
    $password = $_POST['password'];
    $confirm = $_POST['confirm'];

    if($password == $confirm)
    {
        $sql = "INSERT INTO users
                (firstname, lastname, email, username, password)
                VALUES
                ('$fname','$lname','$email','$username','$password')";

        mysqli_query($conn,$sql);

        $message = "Registration Successful!";
    }
    else
    {
        $message = "Password and Confirm Password are not the same.";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Registration</title>
    <link rel="stylesheet" href="style.css">
```

```

</head>
<body>
<div class="container">
<h2>Registration</h2>
<form method="POST">
<input type="text" name="fname" placeholder="First Name" required>
<input type="text" name="lname" placeholder="Last Name" required>
<input type="email" name="email" placeholder="Email" required>
<input type="text" name="username" placeholder="Username" required>
<input type="password" name="password" placeholder="Password" required>
<input type="password" name="confirm" placeholder="Confirm Password" required>
<button type="submit" name="register">
Register
</button>
</form>
<p><?php echo $message; ?></p>
<p>
<a href="login.php">Go to Login</a>
</p>
</div>
</body>
</html>

```

Style.css

```

body{
    background:#f4f4f4;
    font-family:Arial,sans-serif;
}

.container{
    width:450px;
    margin:40px auto;
    background:white;
    padding:20px;
    border-radius:10px;
}
h2{
    text-align:center;
}
input{
    width:100%;
    padding:10px;
    margin-top:10px;
    box-sizing:border-box;
}

```

```

}
button{
  width:100%;
  margin-top:15px;
  padding:10px;
  background:#007bff;
  color:white;
  border:none;
  cursor:pointer;
}
a{
  text-decoration:none;
}

```

- Using your credentials saved in your tables, it will be used to login in the system (create your own login page) include sessions in your webpages.

Registration

Registration

Login

hamsterrrr

.....

Login

User Information

First Name: Ham

Login.php

```
<?php
session_start();
include("db.php");
$message = "";
if(isset($_POST['login']))
{
    $username = $_POST['username'];
    $password = $_POST['password'];

    $sql = "SELECT * FROM users
            WHERE username='$username'
            AND password='$password'";
    $result = mysqli_query($conn,$sql)
    if(mysqli_num_rows($result) > 0)
    {
        $_SESSION['username'] = $username;
        header("Location: profile.php");
        exit();
    }
    else
    {
        $message = "Invalid Login";
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>Login</h2>
<form method="POST">
<input type="text"
name="username"
placeholder="Username"
required>
<input type="password"
name="password"
placeholder="Password"
required>
```

```
<button type="submit" name="login">
Login
</button>
</form>
<p><?php echo $message; ?></p>
</div>
</body>
</html>
```

Logout.php

```
<?php
session_start();
session_destroy();
header("Location: login.php");
exit();
?>
```

Profile.php

```
<?php
session_start();
include("db.php");

if(!isset($_SESSION['username']))
{
    header("Location: login.php");
    exit();
}

$username = $_SESSION['username'];

$sql = "SELECT * FROM users
        WHERE username='$username'";

$result = mysqli_query($conn,$sql);

$user = mysqli_fetch_assoc($result);
?>

<!DOCTYPE html>
<html>
<head>
```

```
<title>Profile</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>User Information</h2>
<p>
First Name:
<?php echo $user['firstname']; ?>
</p>
<p>
Last Name:
<?php echo $user['lastname']; ?>
</p>
<p>
Email:
<?php echo $user['email']; ?>
</p>
<p>
Username:
<?php echo $user['username']; ?>
</p>
<br>
<a href="reset-password.php">
Reset Password
</a>
<br><br>
<a href="logout.php">
Logout
</a>
</div>
</body>
</html>
```

3. User – Side Retrieval of Record

Reset Password

.....

.....

.....

Reset Password

[Back to Profile](#)

Reset Password

Current Password

New Password

Re-enter New Password

Reset Password

Password Updated Successfully!

[Back to Profile](#)

Reset Password

Current Password

New Password

Re-enter New Password

Reset Password

New password and ReEnter new password should be the same.

[Back to Profile](#)

Reset-password.php

```
<?php
session_start();
include("db.php");
if(!isset($_SESSION['username']))
{
    header("Location: login.php");
    exit();
}
$message = "";
if(isset($_POST['reset']))
{
    $username = $_SESSION['username'];
    $current = $_POST['current'];
    $new = $_POST['new'];
    $renew = $_POST['renew'];
    $sql = "SELECT * FROM users
        WHERE username='$username'";
    $result = mysqli_query($conn,$sql);
    $user = mysqli_fetch_assoc($result);
    if($current != $user['password'])
    {
        $message =
            "Current password is not the same with the old password";
    }
    elseif($new != $renew)
    {
        $message =
            "New password and ReEnter new password should be the same.";
    }
    else
    {
        $update =
            "UPDATE users
            SET password='$new'
            WHERE username='$username'";
        mysqli_query($conn,$update);
        $message = "Password Updated Successfully!";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Reset Password</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
<div class="container">
<h2>Reset Password</h2>
<form method="POST">
<input
type="password"
name="current"
placeholder="Current Password"
required>
<input
type="password"
name="new"
placeholder="New Password"
required>
<input
type="password"
name="renew"
placeholder="Re-enter New Password"
required>
<button type="submit" name="reset">
Reset Password
</button>
</form>
<p><?php echo $message; ?></p>
<a href="profile.php">
Back to Profile
</a>
</div>
</body>
</html>
```

4. Screenshot the result of your database from the XAMPP.

← Server: 127.0.0.1 » Database: sa3_db » Table: users

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#)

✓ Showing rows 0 - 1 (2 total, Query took 0.0005 seconds.)

`SELECT * FROM `users``

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows: Sort by key:

Extra options

					id	firstname	lastname	email	username	password		
☐		Edit		Copy		Delete	1	Francine	Agalabia	feagalabia@fit.edu.ph	pransineeeee	12345
☐		Edit		Copy		Delete	2	Ham	Ster	hamster@fit.edu.ph	hamsterrrr	hamster

VII. QUESTION AND ANSWER

1. What are Super Global Variables?

Super Global Variables are predefined PHP variables that can be used anywhere in a PHP script.

2. Differentiate \$_GET, \$_POST, and \$_REQUEST

\$_GET sends data through the URL.

\$_POST sends data in the background and is more secure.

\$_REQUEST can get data from both \$_GET and \$_POST.

3. What is Session?

A Session is used to store user information on the server while the user is using a website.

4. What is a Cookie?

A Cookie is a small file stored in the user's browser to remember information.

5. What is the importance of validation?

Validation helps ensure that the data entered by users is correct and safe.

6. What is database?

A database is a collection of organized data that can be stored, managed, and retrieved easily.

7. What is the importance of database?

A database helps store data securely and makes it easy to search, update, and manage information.

8. What is the good structure of a database?

A good database structure has organized tables, proper field names, and avoids duplicate data.

VIII. REFERENCES

1. <https://www.w3schools.com/css/>
2. <https://www.w3schools.com/html/>
3. https://www.w3schools.com/php/php_variables.asp
4. https://www.w3schools.com/php/php_superglobals_globals.asp
5. https://www.w3schools.com/php/php_superglobals_post.asp
6. https://www.w3schools.com/php/php_superglobals_get.asp
7. https://www.w3schools.com/php/php_superglobals_request.asp
8. https://www.w3schools.com/php/php_cookies.asp
9. https://www.w3schools.com/php/php_sessions.asp
10. https://www.w3schools.com/php/php_mysql_intro.asp
11. https://www.w3schools.com/php/php_mysql_connect.asp

12. https://www.w3schools.com/php/php_mysql_create.asp
13. https://www.w3schools.com/php/php_mysql_create_table.asp
14. https://www.w3schools.com/php/php_mysql_insert.asp
15. https://www.w3schools.com/php/php_mysql_insert_multiple.asp
16. https://www.w3schools.com/php/php_mysql_select.asp
17. https://www.w3schools.com/php/php_mysql_select_where.asp
18. https://www.w3schools.com/php/php_mysql_select_orderby.asp
19. https://www.w3schools.com/php/php_mysql_delete.asp
20. https://www.w3schools.com/php/php_mysql_update.asp
21. <https://skillforge.com/how-to-create-a-database-using-phpmyadmin-xampp/>

Note: The following rubrics/metrics will be used to grade students' output.

Program (100 pts.)	(Excellent)	(Good)	(Fair)	(Poor)
Program execution (20pts)	Program executes correctly with no syntax or runtime errors (18-20pts)	Program executes with less than 3 errors (15-17pts)	Program executes with more than 3 errors (12-14pts)	Program does not execute (10-11pts)
Correct output (20pts)	Program displays correct output with no errors (18-20pts)	Output has minor errors (15-17pts)	Output has multiple errors (12-14pts)	Output is incorrect (10-11pts)
Design of output (10pts)	Program displays more than expected (10pts)	Program displays minimally expected output (8-9pts)	Program does not display the required output (6-7pts)	Output is poorly designed (5pts)
Design of logic (20pts)	Program is logically well designed (18-20pts)	Program has slight logic errors that do no significantly affect the results (15-17pts)	Program has significant logic errors (3-5pts)	Program is incorrect (10-11pts)
Standards (20pts)	Program code is stylistically well designed (18-20pts)	Few inappropriate design choices (i.e. poor variable names, improper	Several inappropriate design choices (i.e. poor variable names, improper	Program is poorly written (10-11pts)

		indentation) (15-17pts)	indentation) (12-14pts)	
Delivery (10pts)	The program was delivered on time. (10pts)	The program was delivered a day after the deadline. (8-9pts)	The program was delivered two days after the deadline. (6-7pts)	The program was delivered more than two days after the deadline. (5pts)